

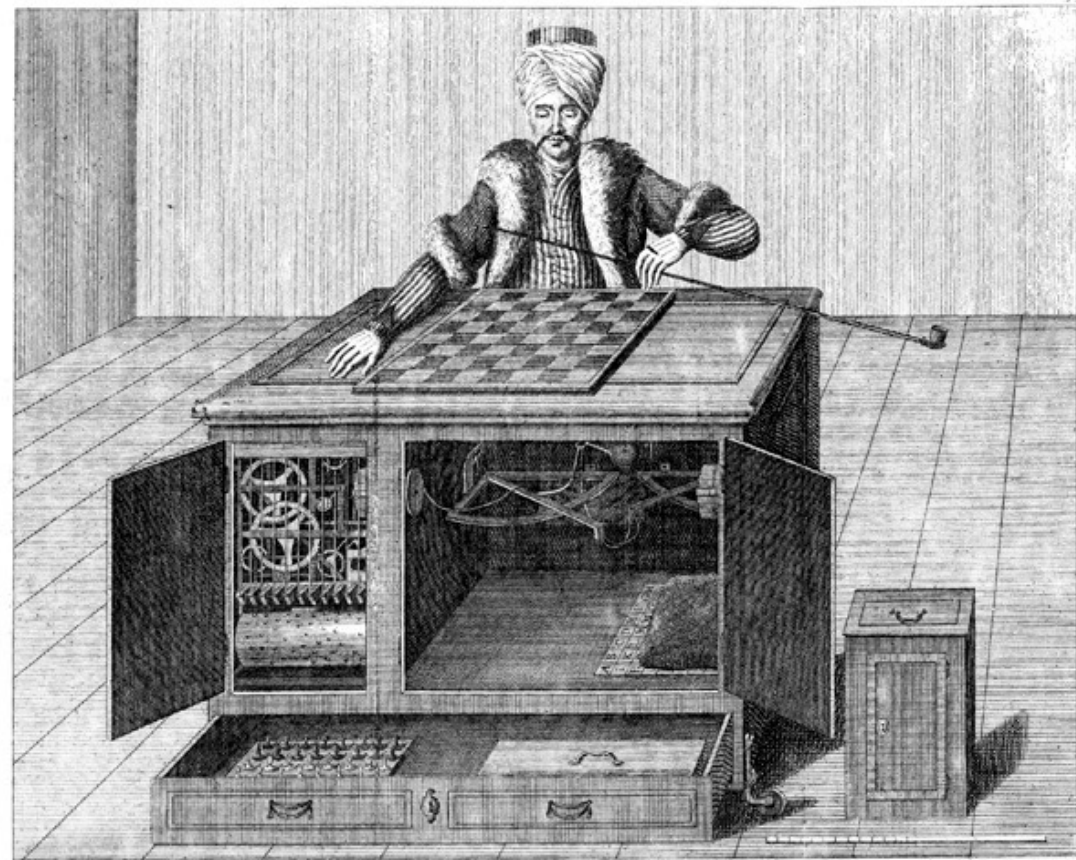
Game-playing Systems

fredrik.johansson@chalmers.se

February 20, 2024

The Mechanical Turk (Not actually AI...)

- 1700s



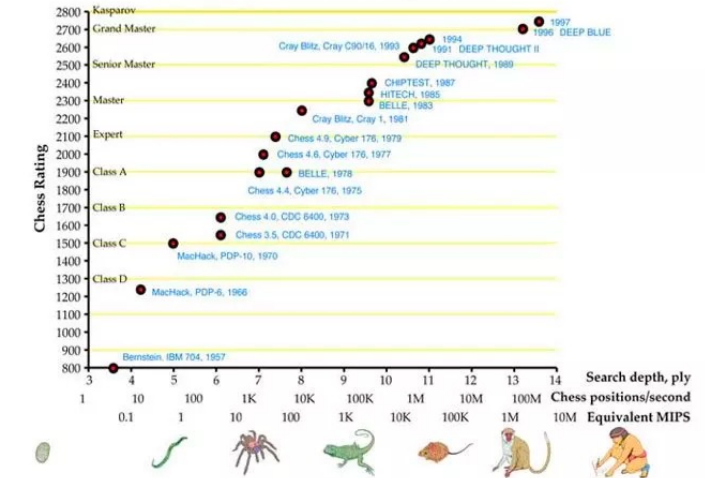
W. de Kempelen del. Ch. a. Mechel excud. Basilea. P.G. Pütz, fec.
Der Schach-Spieler, wie er vor dem Spiele gezeigt wird von vorn. Le Joueur d'Echecs, tel qu'on le montre avant le jeu, par devant.

Chess computers

- 1900s



Chess Machine Performance versus Processing Power



DoTA

- 2017: OpenAI Five



<https://youtu.be/pkGa8ICQJS8?t=845>

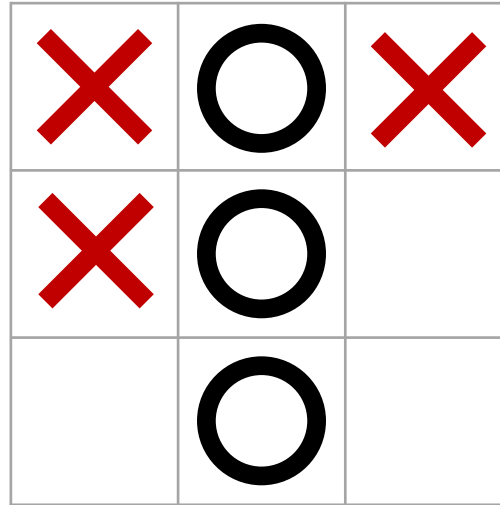
Outline

- Formalizing games
 - Zero-sum games
 - Tree search
 - Games as decision processes
- Learning to play
 - Monte-Carlo tree search
 - Exploration/exploitation
 - Machine learning

Formalizing games

Zero-sum games: Tic-Tac-Toe

- **Zero-sum:** One player winning implies one player losing



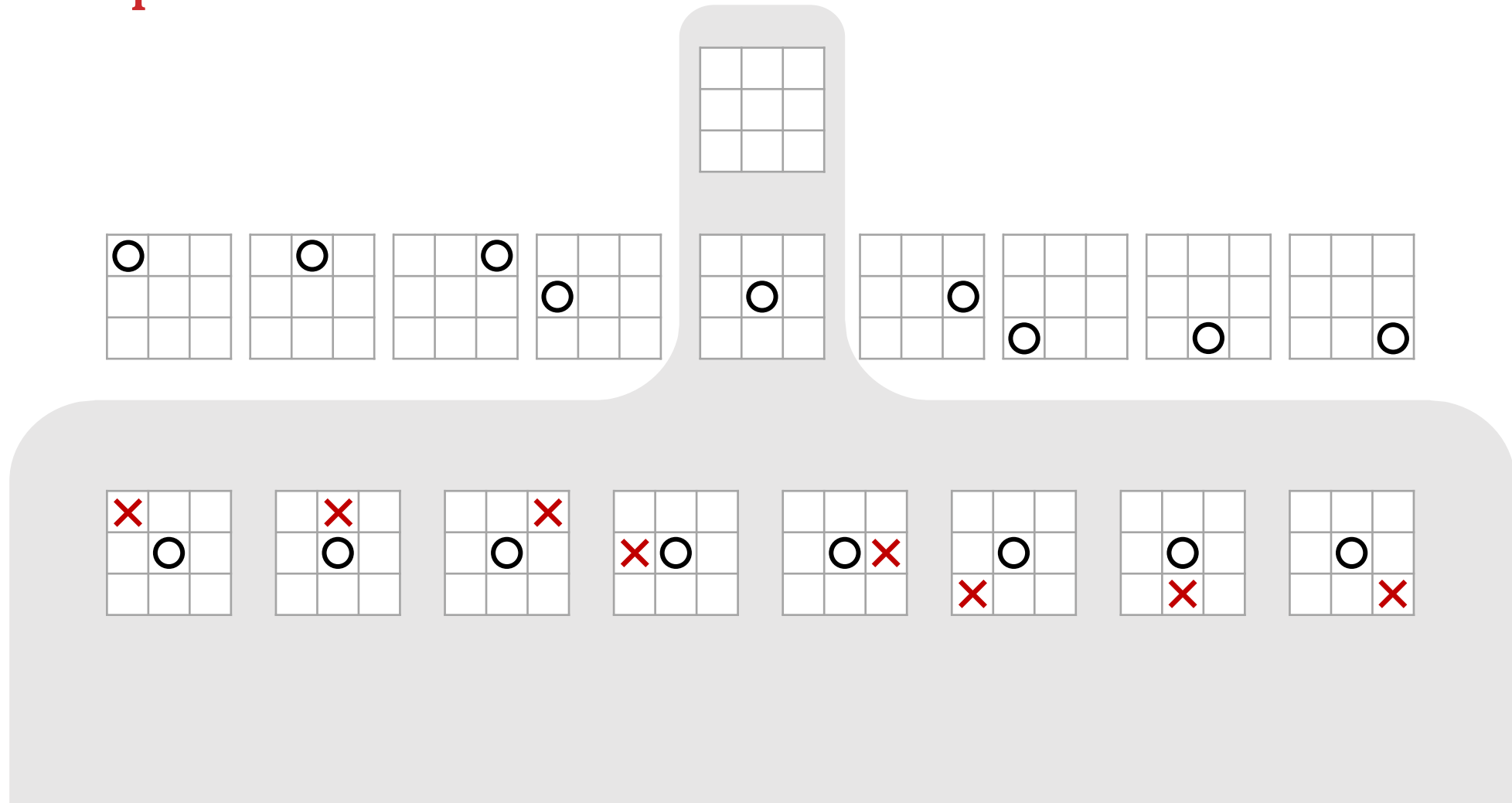
- **Example:** The winning player receives reward $R = +1$ and the losing player $R = -1$. The sum is 0.

How do we win at games?

- Games are inherently about **actions**
- Our goal is to select actions that improve our **chances of winning**
- To win, we account for **good/bad futures** when selecting actions

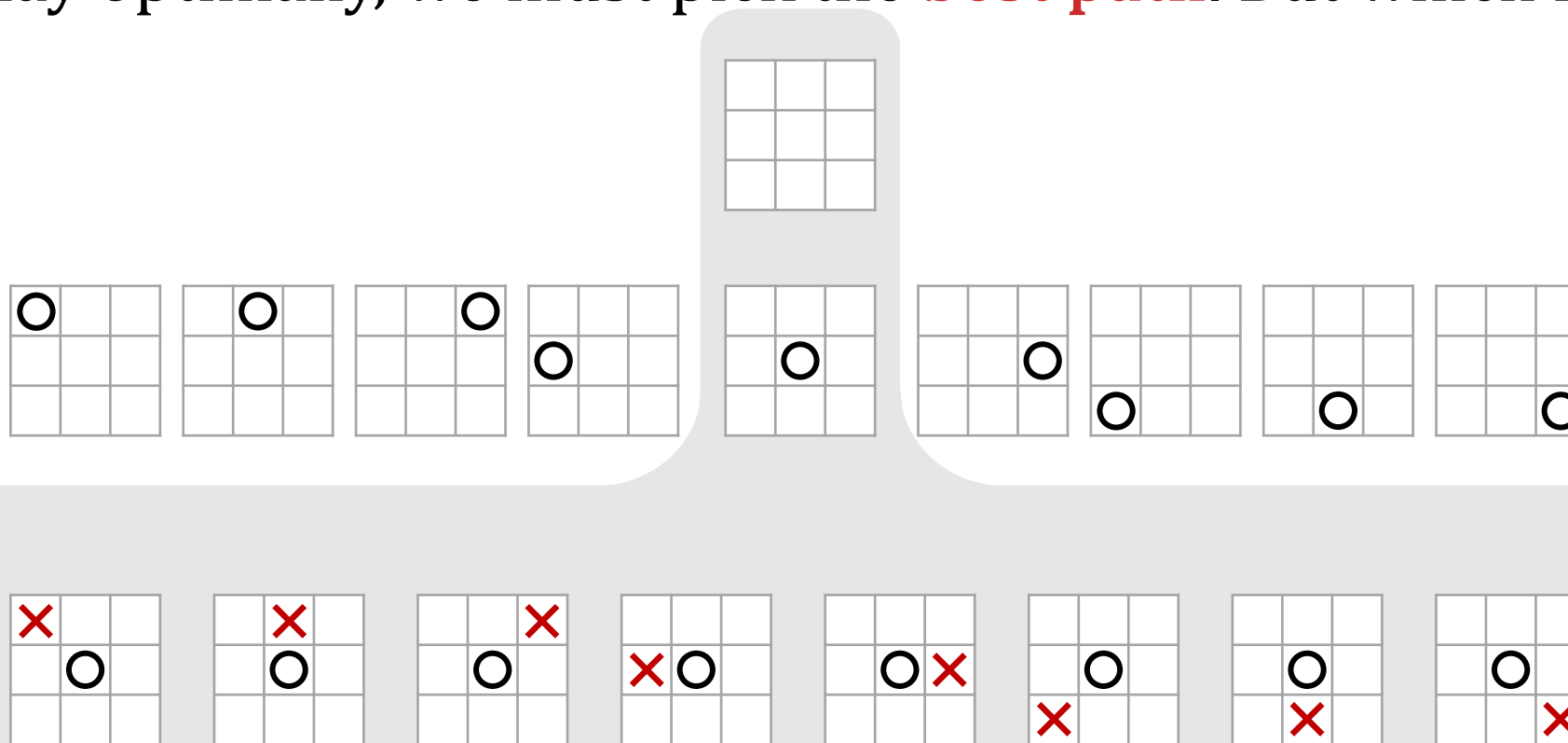
Branching paths

- **Example:** Tic-Tac-Toe

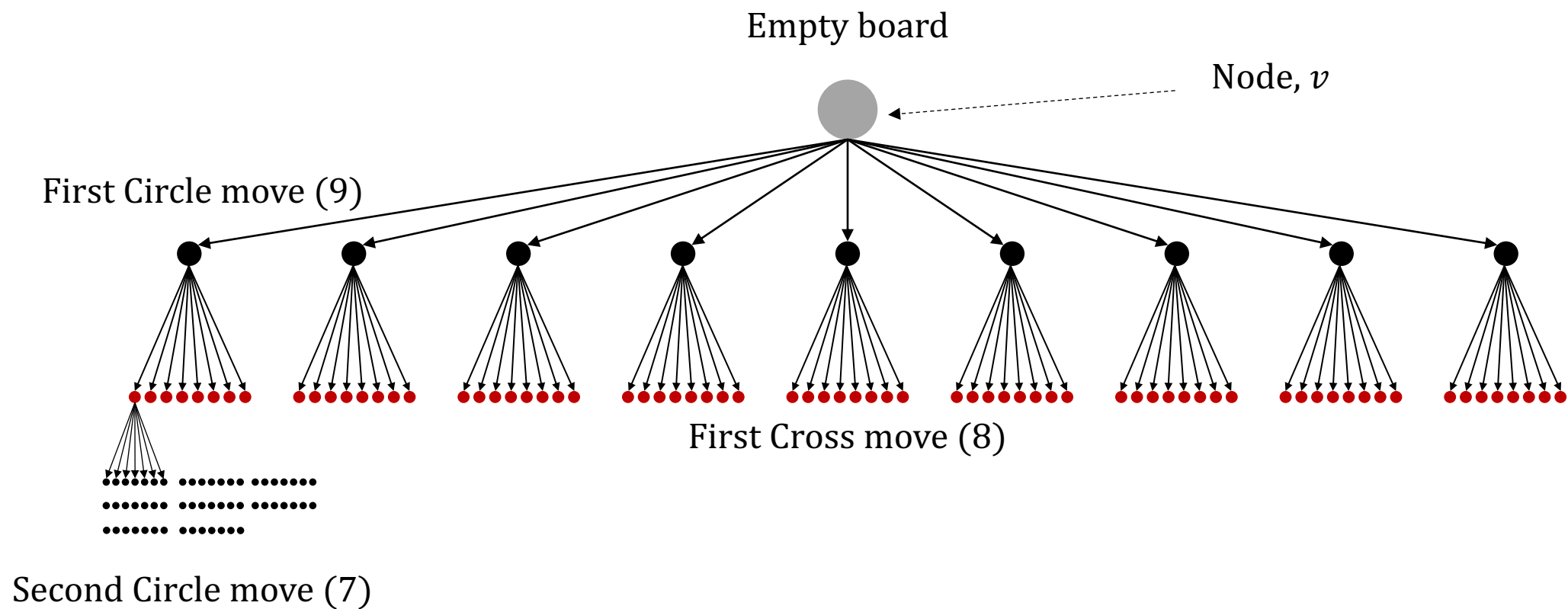


Branching paths

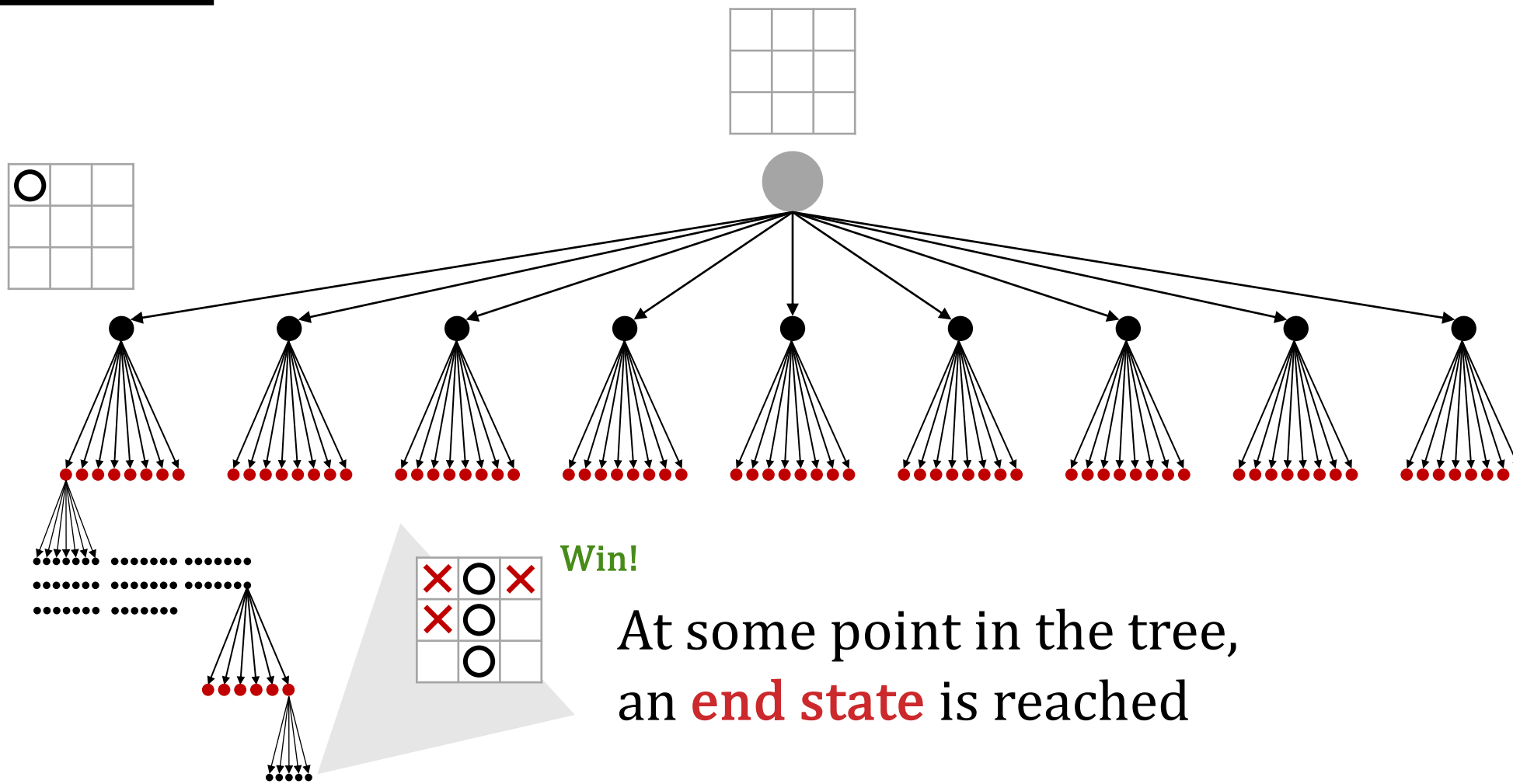
- To play optimally, we must pick the **best path**. But which is it?



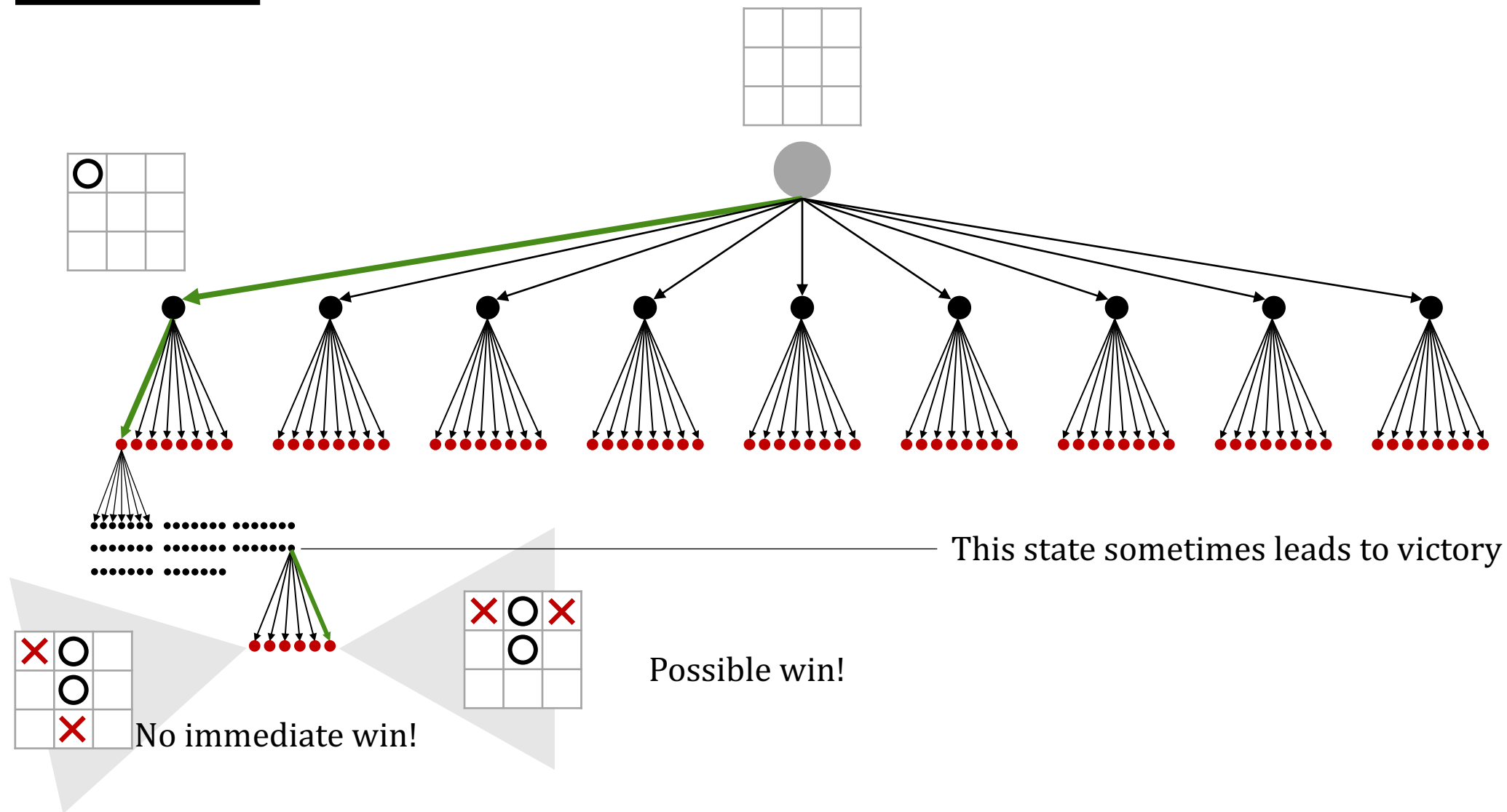
Formalization: Search trees



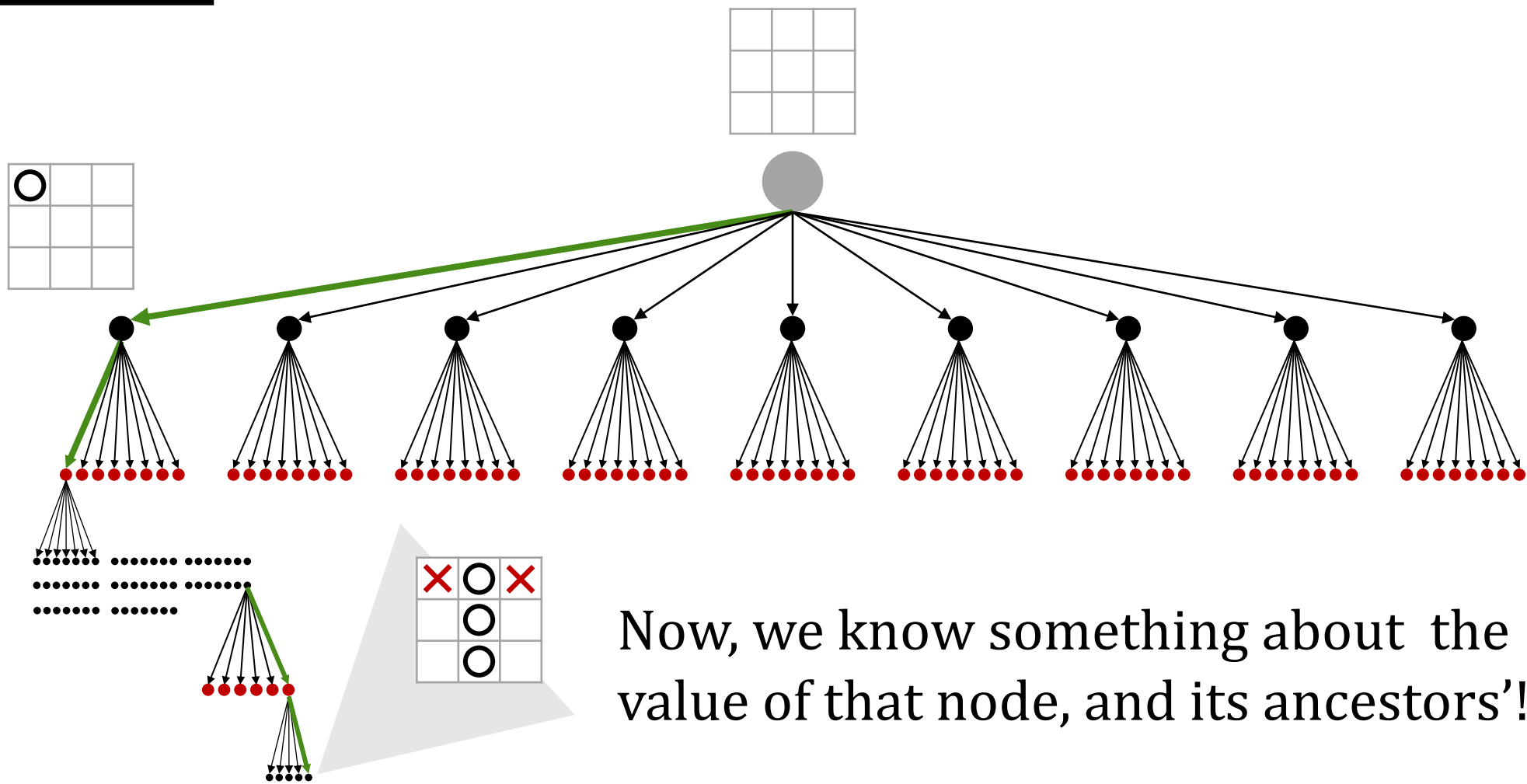
Terminal nodes



Value of state

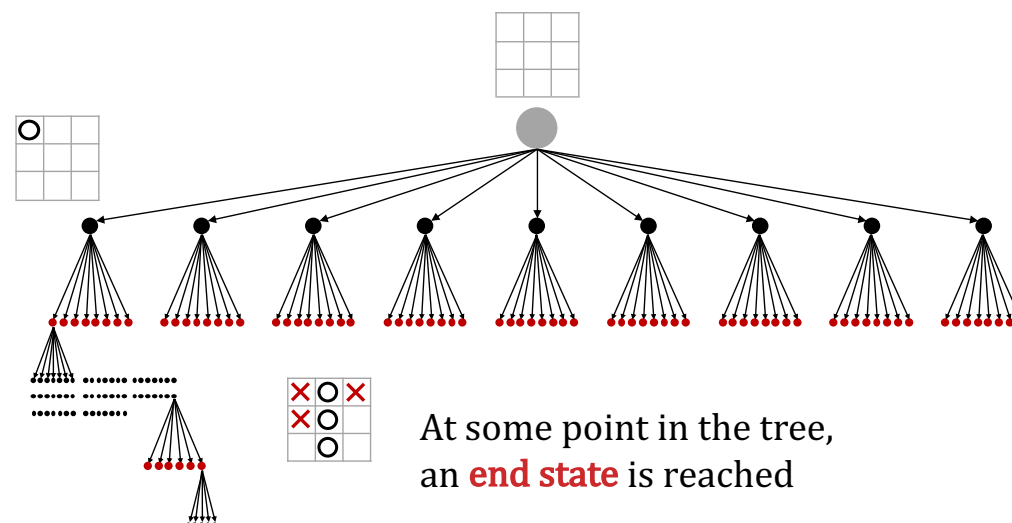


Value of state



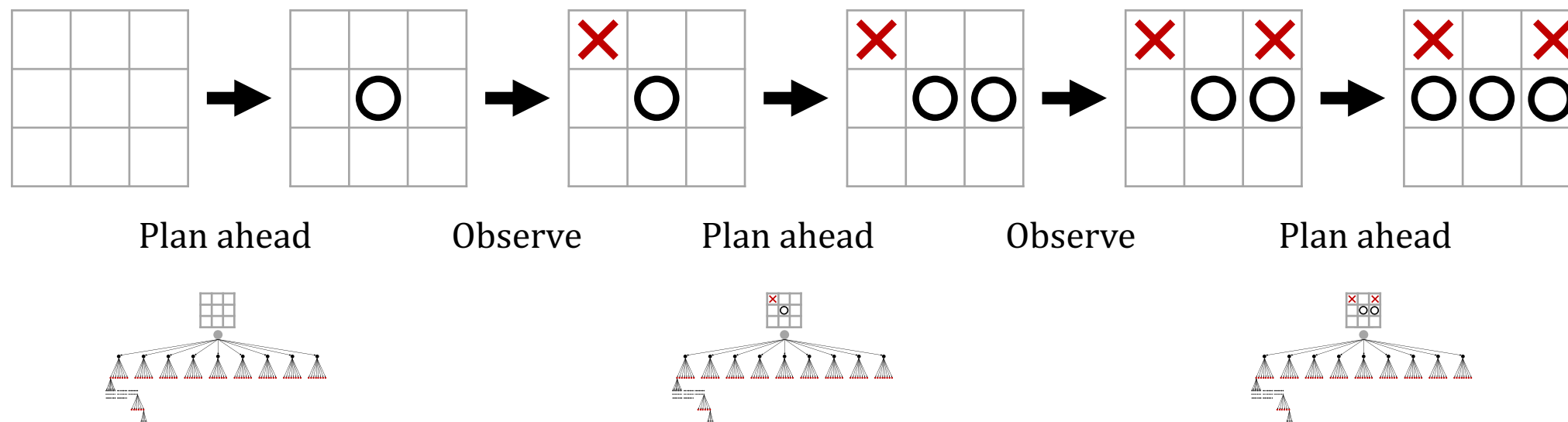
Search trees

- A search tree can be used to enumerate **possible futures**
- By identifying good futures (wins), we can **backtrack**
— **Which actions** do we have to take to get to the good end?

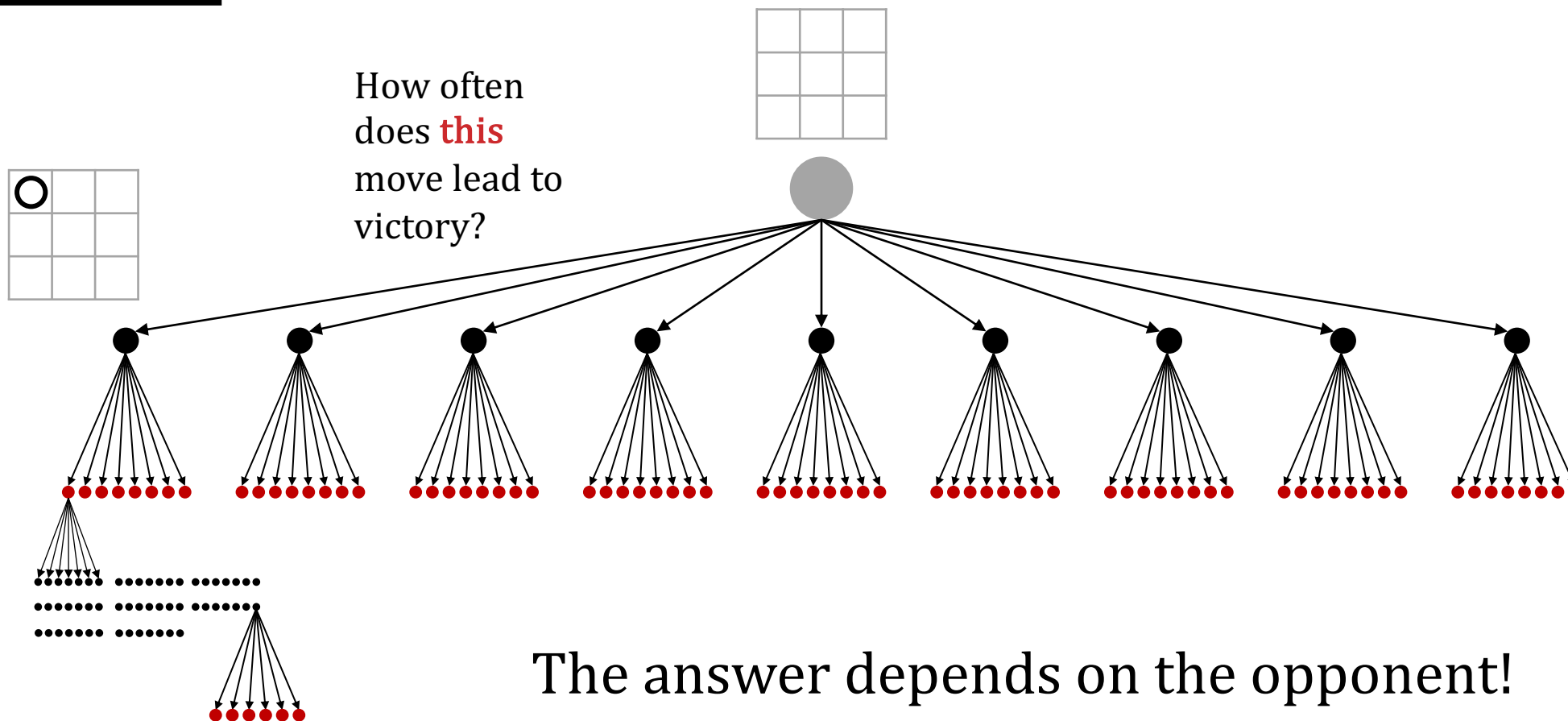


Playing using search trees

- A plan is **only a plan**—you don't control your opponent



Value of state

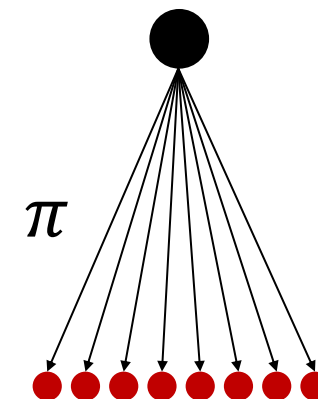


Playing in a stationary environment

- Assume for now that your opponent has a **fixed, known policy, π**

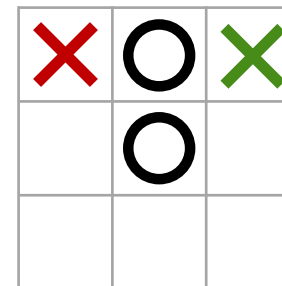
For each position

1. Compute the probability of reaching winning states
 - Based on the probability of future states
2. Execute the action(s) with the highest success rate

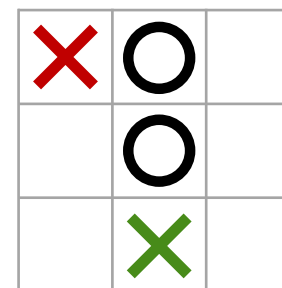


What if your opponent adapts?

- If your opponent **used to be** suboptimal ...
- But changes to being optimal, the probabilities of next states **change** — and so does your win probability



Suboptimal Cross



Optimal Cross

Minimax optimization

- To guarantee success against the best possible opponent:
Minimize the maximum success of your opponent (worst case)
- Assume that you knew the best possible move and that your opponent will play that move. Then proceed as before:
 - Count future success rate of each action, and act accordingly
- But **what is the best** move?

Games as decision processes

- For our purposes, a decision process describes an **agent**
 - taking **actions** A_t
 - according to a **policy**, π (π maps an observation X_t to an action A_t)
 - in **states** S_t , **observed** through X_t ,
 - which transition according to **dynamics** $p(S_t \mid S_{<t}, A_{<t})$
- Often, we have a designated **reward** R_t , a function of X_t that we want to optimize. For example, $R_t = +1$ for winning, -1 for losing

Minimax games

- In two-player games, there are two actors with (potentially) **different policies**, π (player), μ (opponent)
- $\bar{R}(\pi, \mu)$ denotes the average **reward** (e.g., win rate) of π versus μ
- Minimax play for π against μ **optimizes**

$$\min_{\pi} \underbrace{\max_{\mu} \bar{R}(\mu, \pi)}_{\text{Strongest opponent}}$$

Minimax planning

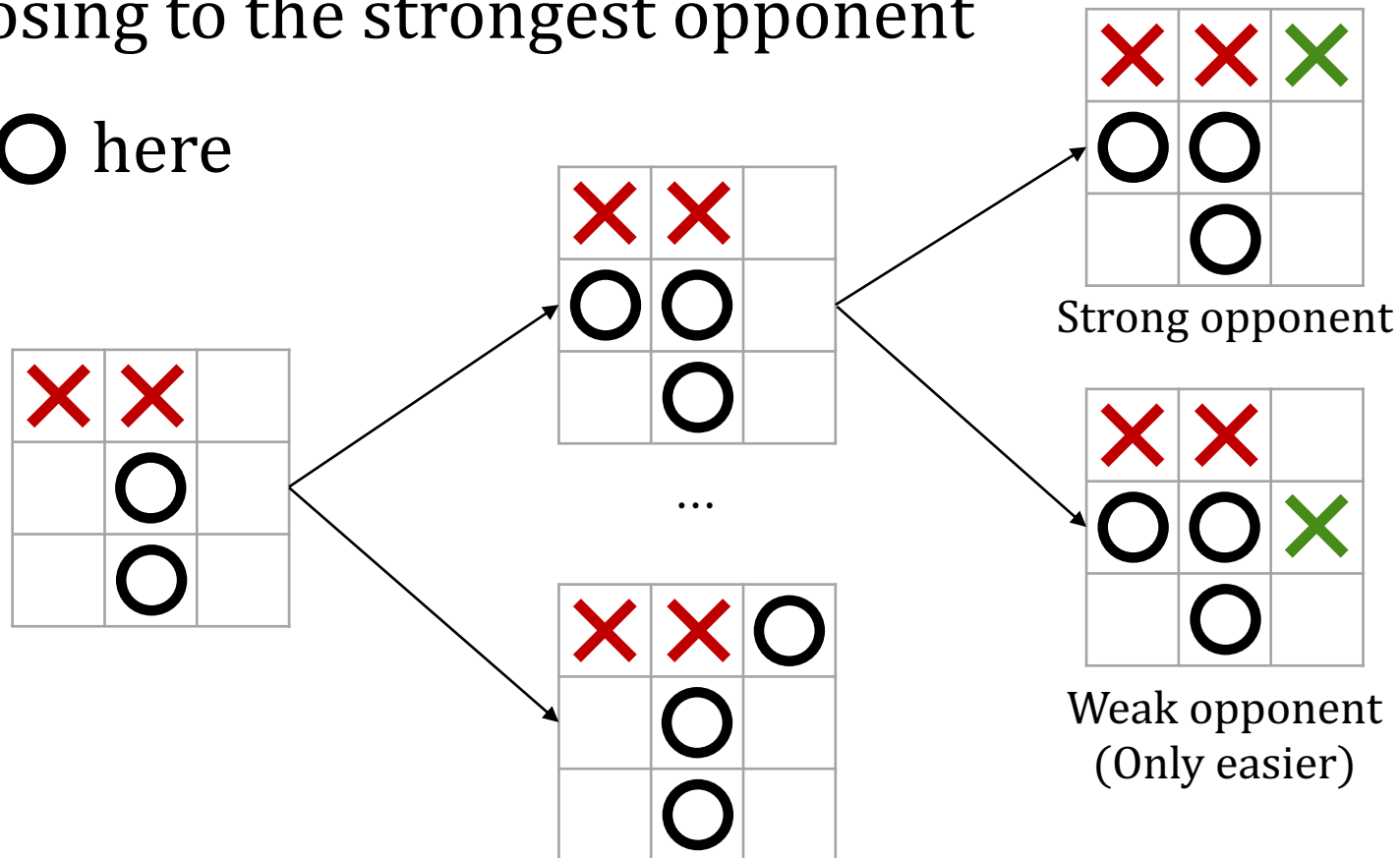
- In search trees, minimax planning selects the next action at time t by examining the best **path** $A_t, \dots, A_T$ for the player and the best path $B_{t+1}, \dots, B_T$ for the opponent until end T

$$\min_{A_t, \dots, A_T} \max_{B_{t+1}, \dots, B_T} \bar{R}(A, B)$$

- Which move has the smallest probability of losing against an optimal opponent, if we keep playing optimally after this move?

Minimax: Avoiding losses

- Minimax avoids losing to the strongest opponent
- We're playing as ○ here



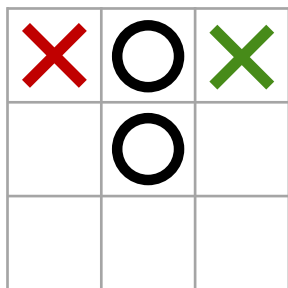
Discuss

What are potential drawbacks of minimax optimization?

Example games

- Actions vary, rewards for winning

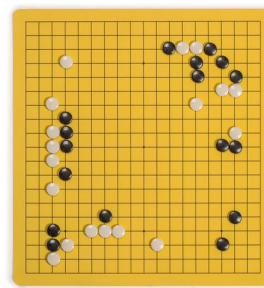
Fully observed S_t
Discrete action space



Tic-Tac-Toe

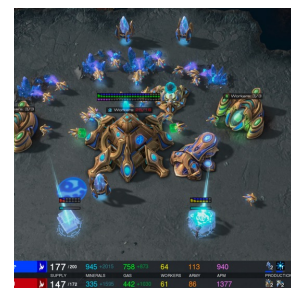


Chess



Go

Partially observed S_t
“Continuous” action space



StarCraft



DoTA 2

1v1

5v5

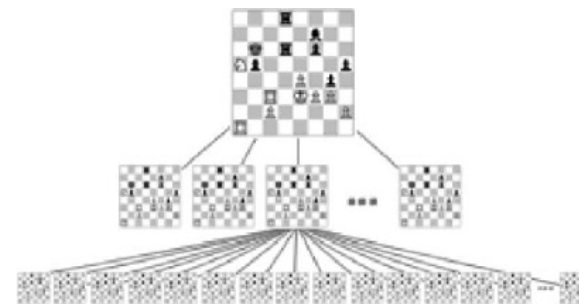
Exhaustive search

- In Tic-Tac-Toe, the number of possible states (boards) $< 20\,000$
 - We can enumerate all of them easily!
- To play minimax optimally, we just have to find the path with the highest win-rate for the opponent, and avoid that

Exhaustive search not always feasible

- In Tic-Tac-Toe, the number of possible states (boards) $< 20\,000$
 - We can enumerate all of them easily!

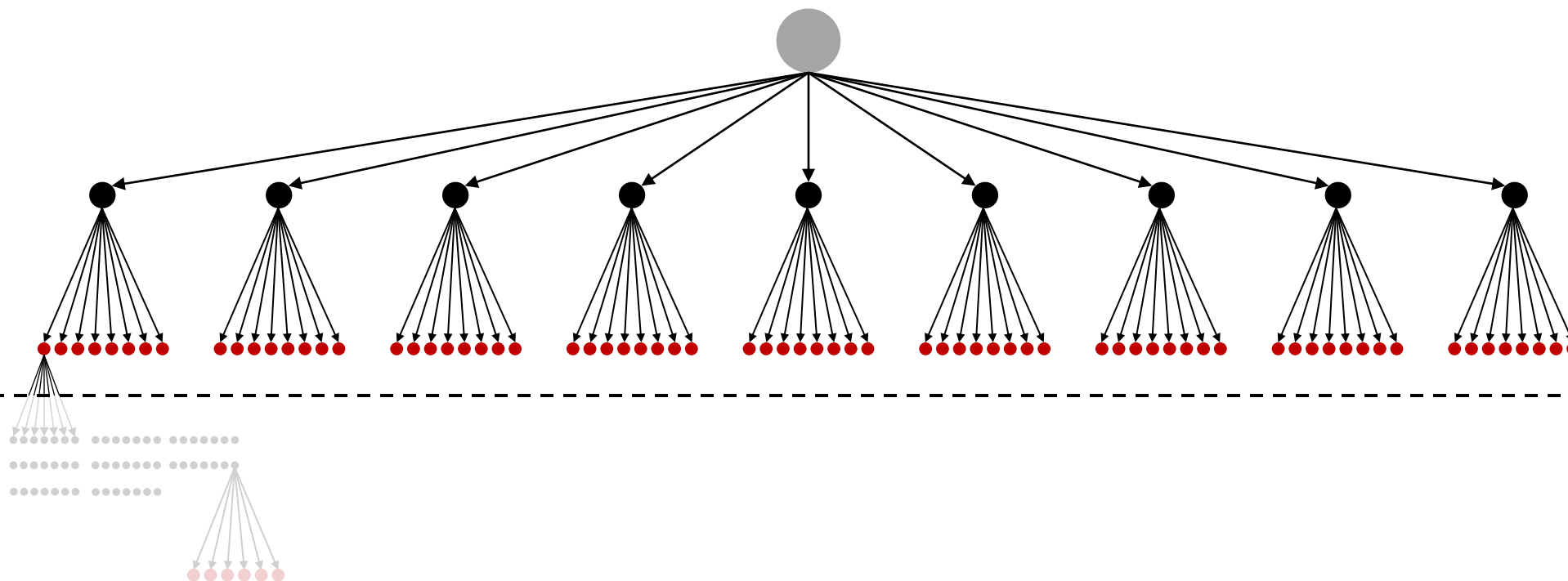
- In Chess, this number is $\sim 10^{47}$.



- We **can't always** enumerate/evaluate all possible states

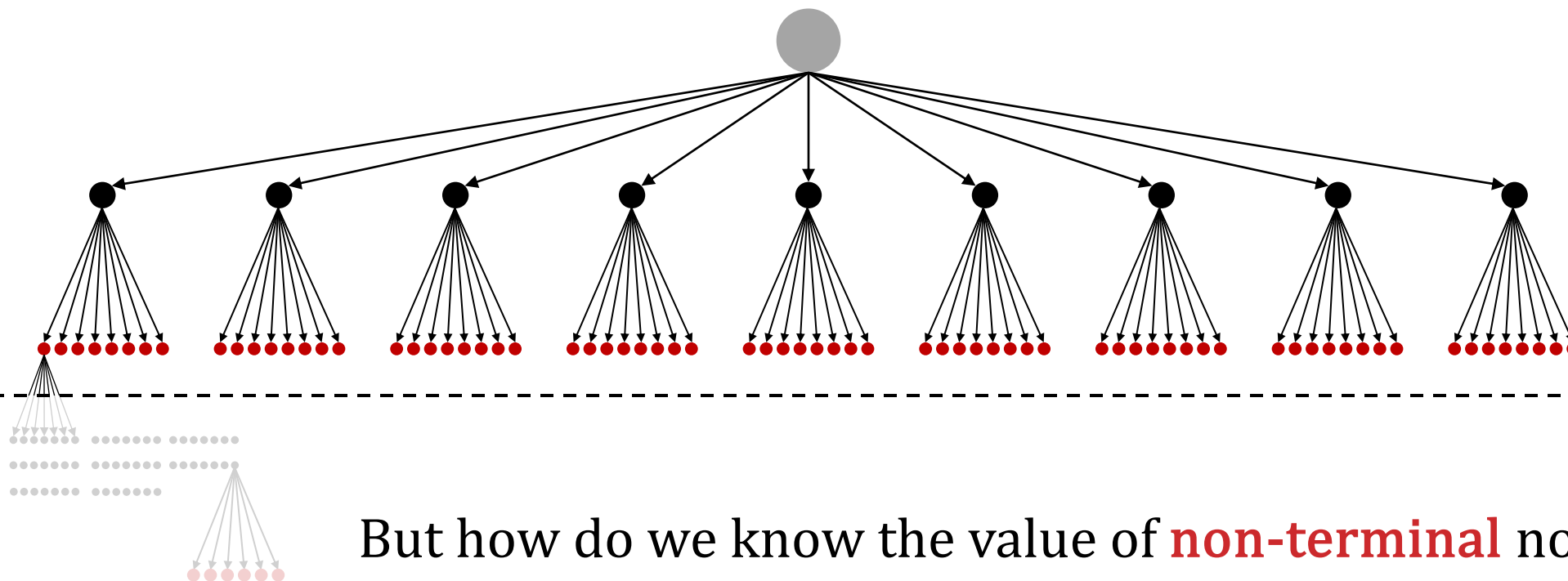
Truncating at finite depth

- We may limit ourselves to looking d steps ahead



Truncating at finite depth

- We may limit ourselves to looking d steps ahead



But how do we know the value of **non-terminal** nodes?

Evaluating non-terminal nodes

- For humans, this is quite natural

- This

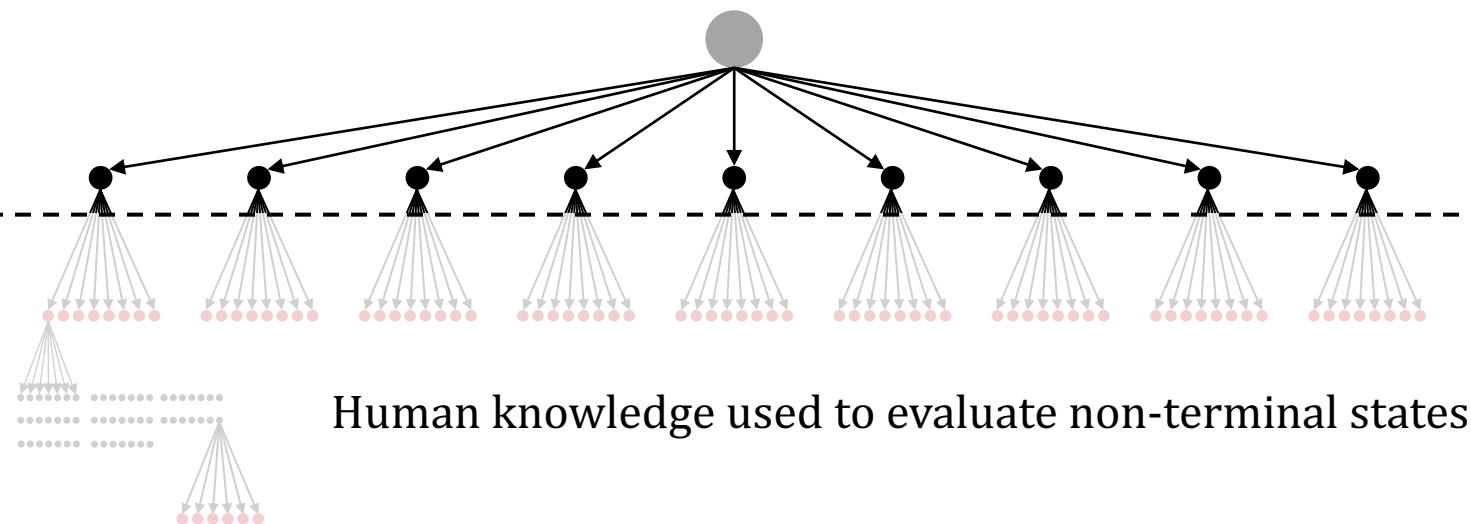
×	×	
○	○	

 is a **bad** state for Cross if it is Circle's turn

- But how do we encode this knowledge in **AI**?

Early game-playing AI

- Early systems used **human knowledge** to evaluate states*
- For example, in early AI for Backgammon and Chess, points were assigned "manually" to non-terminal states



*Also first version of AlphaGo!

Learning to play

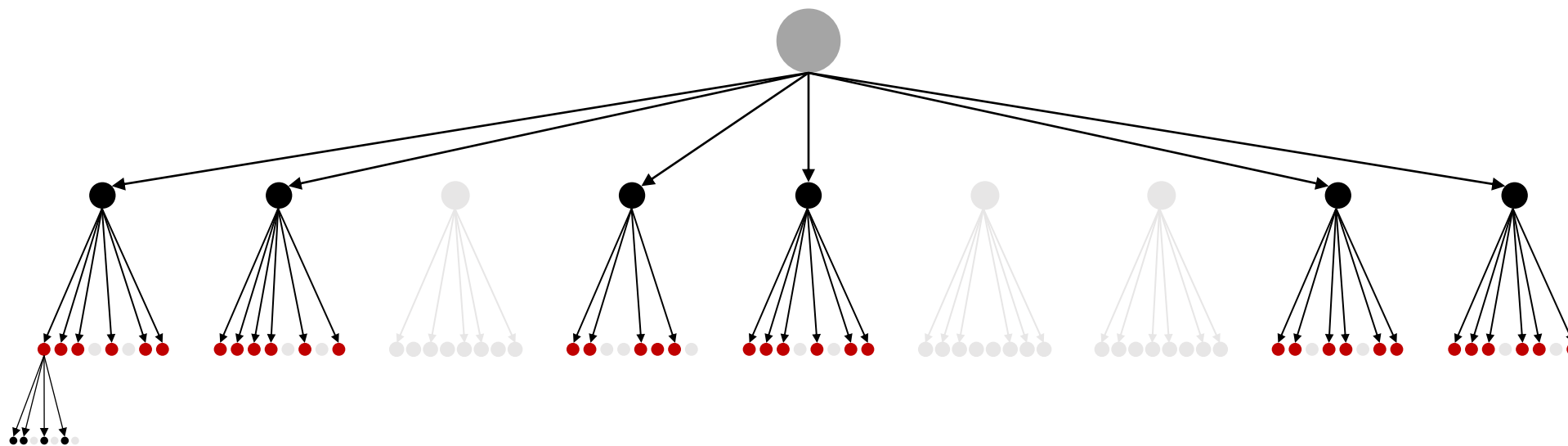
Two problems with large state spaces

1. We **can't explore** every state
 - We won't experience everything in feasible time spans
2. We **can't store** the value of each state
 - A table housing all state-value pairs would be too huge

Two problems with large state spaces

1. We **can't explore** every state
 - We won't experience everything in feasible time spans
 2. We can't store the value of each state
 - A table housing all state-value pairs would be too huge
- Let's start with (1)

What if we try only some actions?



How do we make sure we don't leave out good ones?

Monte-Carlo tree search

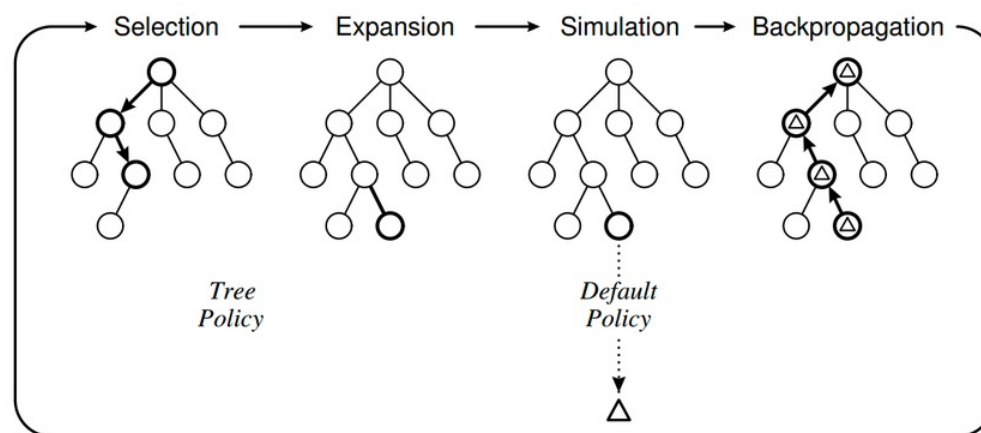
- Monte-Carlo methods deal with **random sampling**
- Instead of searching exhaustively
— search based on experiments with **randomly selected actions**
- Try playing

×	×	○
	○	
	○	

 and **remember** what happened

Monte-Carlo Tree Search

- Monte-Carlo Tree Search (MCTS) implements this idea
- To pick its next move, MCTS **repeats** the following process

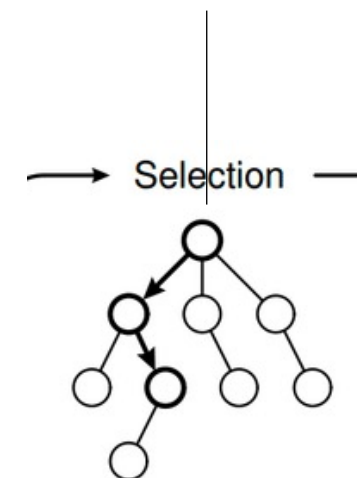


- and in the end, picks its move based on gathered statistics

MCTS 1: Selection

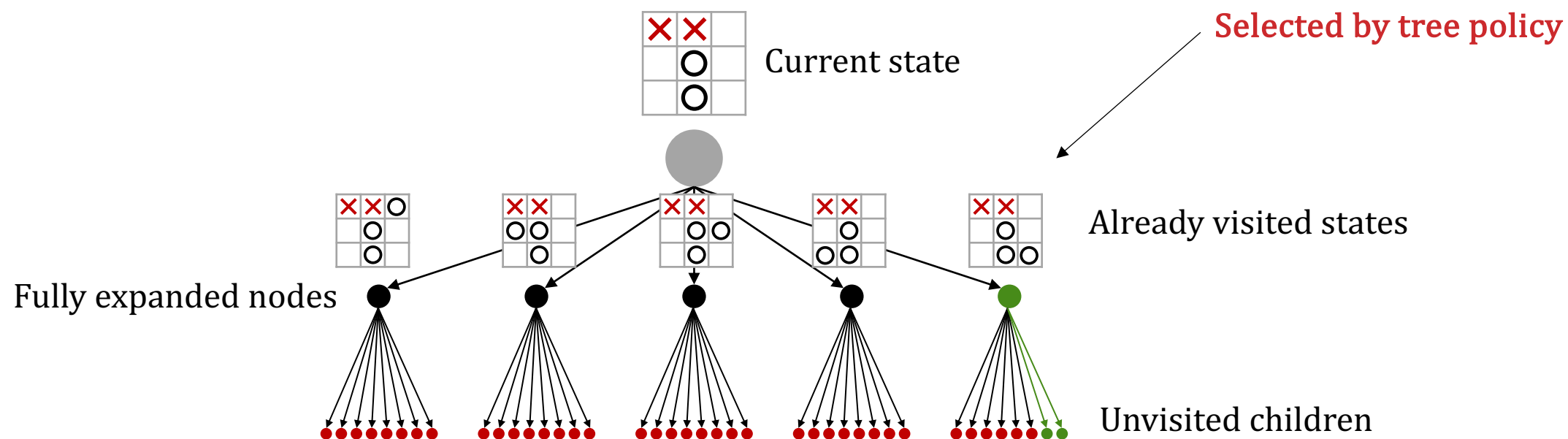
- Selection refers to finding **unexplored** nodes — state-action pairs that haven't been tried
- Based on what we know so far, traverse down until an **unexplored child** node is reached
- The traversal uses a **tree policy** for selection — more on this later

Start from current
Position (root)



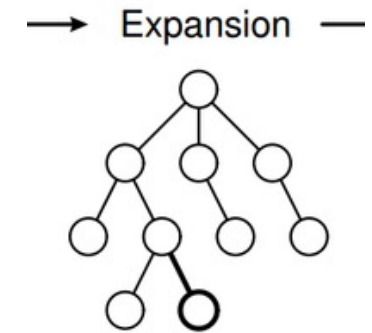
*Tree
Policy*

Selection example



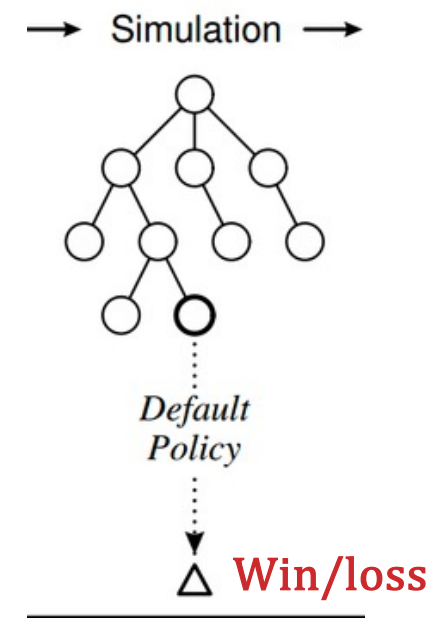
MCTS 2: Expansion

- Once a node has been selected, it is **expanded**
- **An unvisited child node** is selected
- This is critical to avoid leaving out good moves
- We must now **evaluate** this node—how?



MCTS 3: Simulation/roll-out

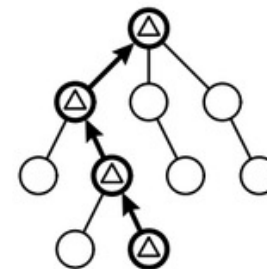
- Once an unvisited child node is selected, we **simulate** the game starting there
- Based on a **simulation policy**, typically a simple random policy, we continue playing from the expansion node until a **terminal node**
- At the terminal node, **we know the value!**



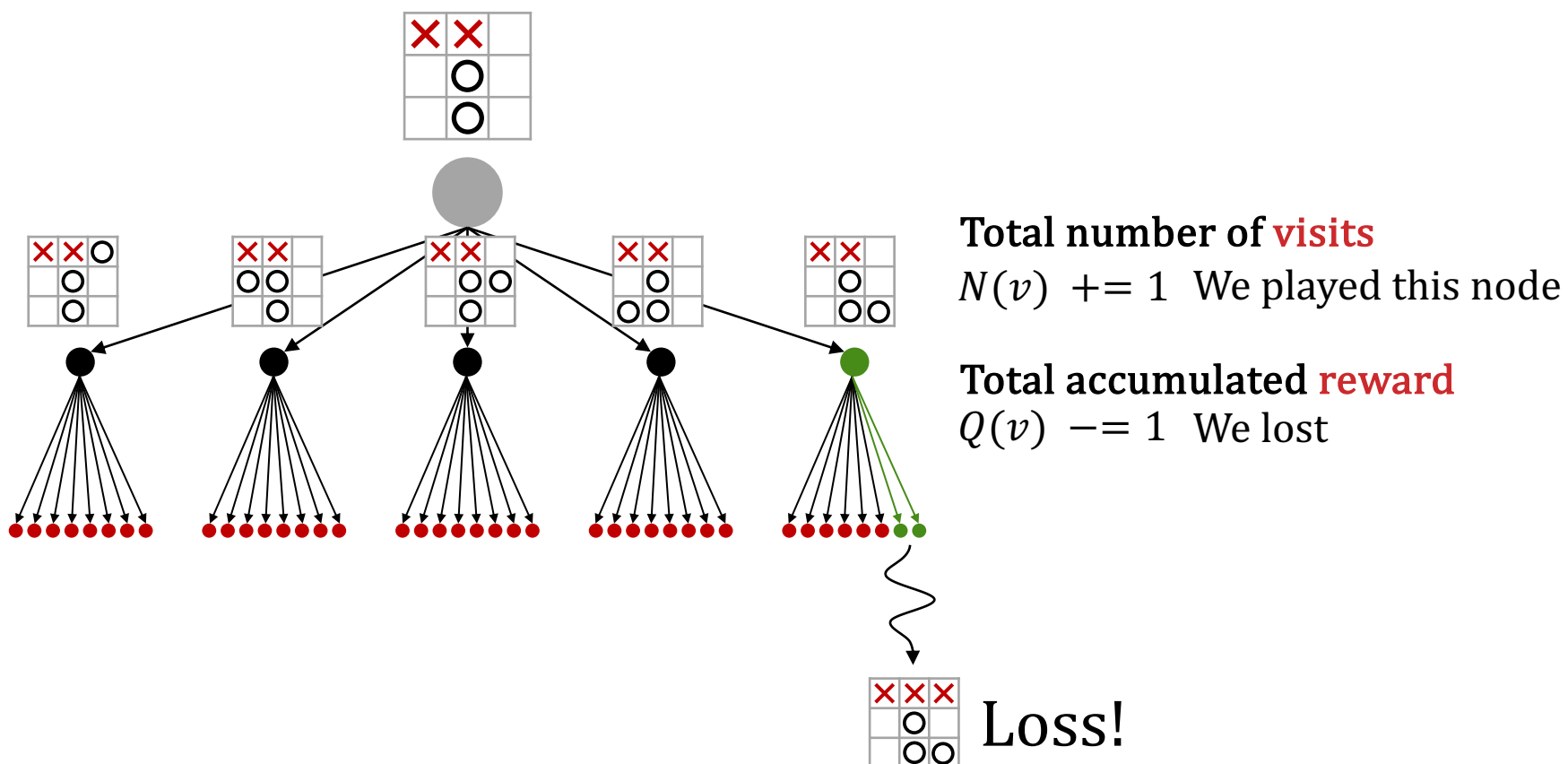
MCTS 4: Backpropagation

- Once we know the value of a terminal node, we **backpropagate** it through the tree
- We **update** our idea of how good moves were
- The expanded node is marked as **visited**
- Statistics of nodes are updated, for example
 - Number of visits, $N(v)$
 - Total reward starting there, $Q(v)$ — the “**Q-value**”

→ Backpropagation ~

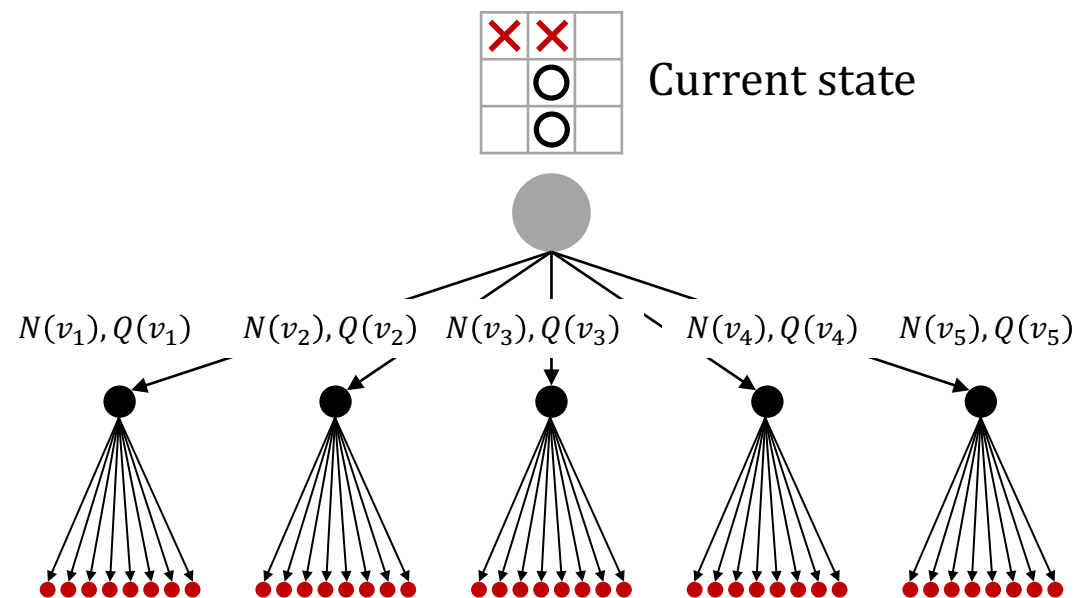


Backpropagation example

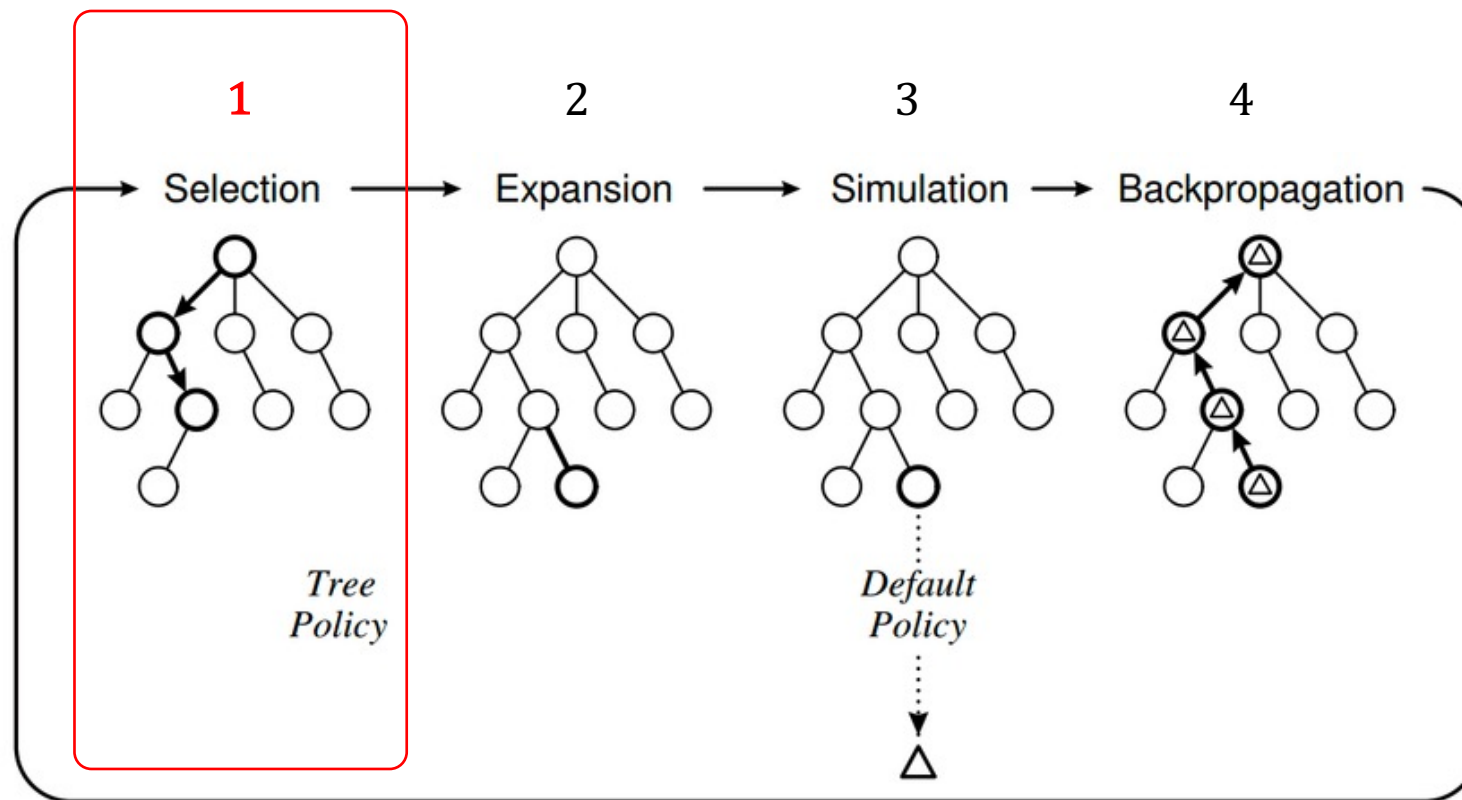


We now know some rewards and #visits

- How do we traverse the tree based on N and Q



Monte-Carlo Tree Search



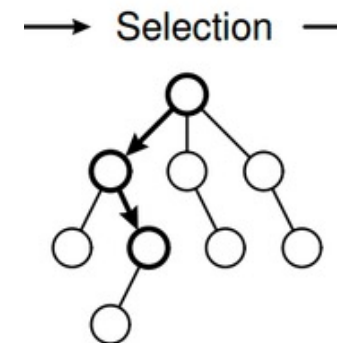
What is a good tree policy?

- Selecting nodes randomly is very inefficient
 - The **statistics** we collect can be used to improve selection during MCTS and the action we choose to play in the end
- How should we **balance** trying new things and using knowledge?

Attempt 1: Greedy policies

- A **greedy** policy selects the **best action** according to some metric
- For example, the highest average reward accumulated so far:

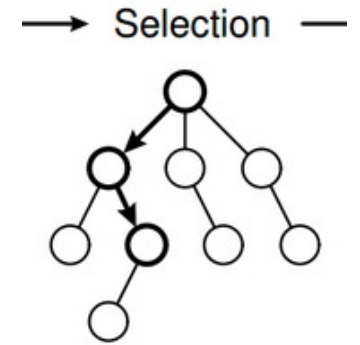
$$A = \arg \max_a \frac{Q(v_a)}{N(v_a)}$$



- **Problem:** This leads to **no exploration**
- This is good when Q is a very good approximation of value

Attempt 2: ϵ -greedy policies

- An ϵ -greedy policy chooses the greedy action with probability $1 - \epsilon$, and a random action with probability ϵ
- Trades off exploration and exploitation **a little**
- **Problem:** May try an action **known to be bad**, by chance
- Can we provably do better?



Attempt 3: Upper confidence bounds

- Upper confidence bounds (UCB) is a popular heuristic
- Bound on the true Q -value for an action a in node v

$$Q(v_a) \leq \hat{Q}_t(v_a) + \hat{U}_t(v_a)$$

Estimated value

True value
Uncertainty

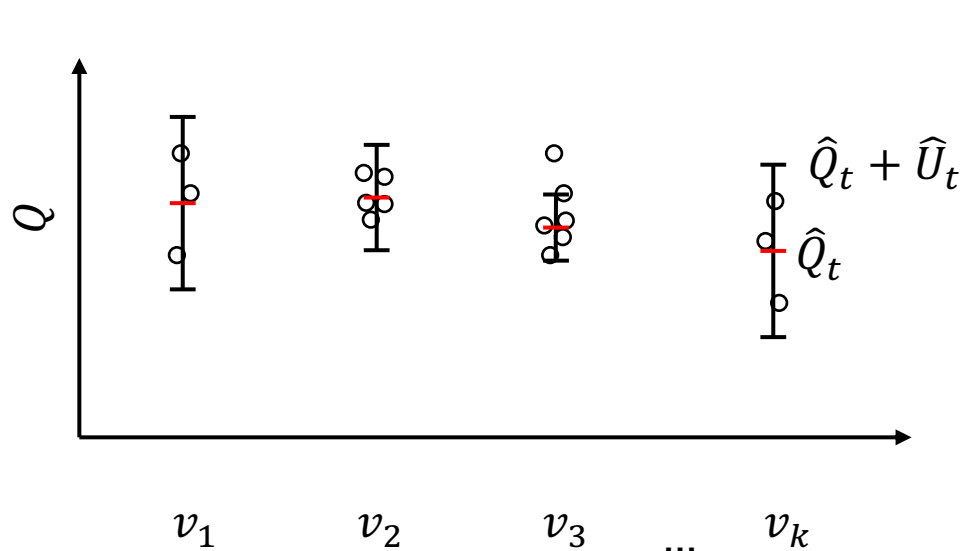
What is this?

- and acts greedily: $a_t = \arg \max_a [\hat{Q}_t(v_a) + \hat{U}_t(v_a)]$

Upper confidence bounds: motivation

- UCB bound on the Q value is based on concentration inequalities

With high probability: $Q(v_a) \leq \hat{Q}_t(v_a) + \hat{U}_t(v_a)$



$$\hat{U}_t(v_a) := c \sqrt{\frac{\log N(v)}{N(v_a)}}$$

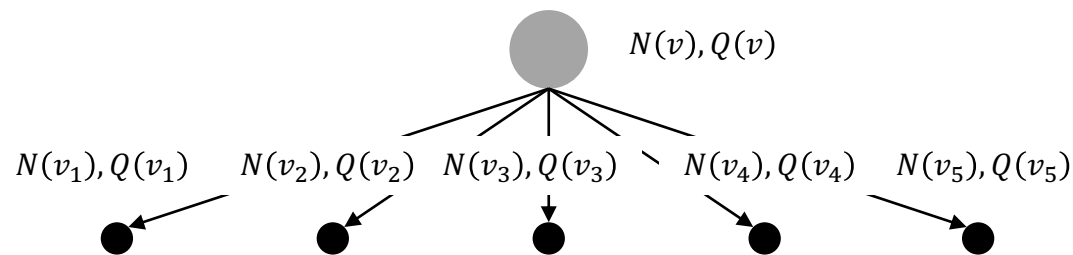
From Hoeffding's ineq.

The more samples we see, the more sure we can be that a certain action is not optimal

Upper confidence trees (UCT)

- The application of UCB to search trees is called UCT
- One variant is defined as follows (but others exist)

$$\text{UCT}(v_i, v) = \frac{Q(v_i)}{N(v_i)} + c \sqrt{\frac{\log N(v)}{N(v_i)}}$$



Upper confidence trees (UCT)

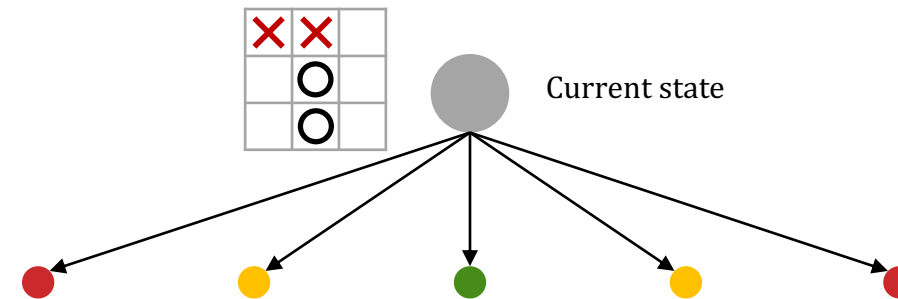
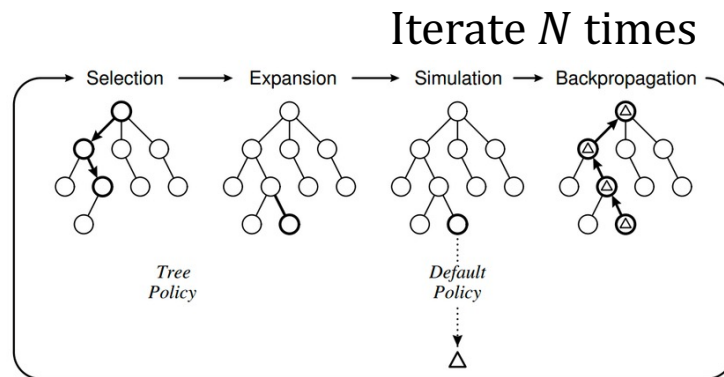
- The application of UCB to search trees is called UCT
- One variant is defined as follows (but others exist)

$$\operatorname{argmax}_{v_i} \quad \text{UCT}(v_i, v) = \frac{Q(v_i)}{N(v_i)} + c \sqrt{\frac{\log N(v)}{N(v_i)}}$$

Child nodes i with small $N(v_i)$ preferred!

Finishing the search

- MCTS is repeated until sufficient statistics have been gathered
- Each potential action a has statistics $N(v_a)$, $Q(v_a)$



- We now **choose the action corresponding to the most visited state**

Two problems with large state spaces

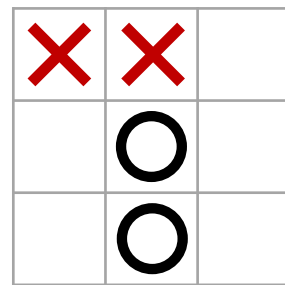
1. We can't explore every state
 - We won't experience everything in feasible time spans
 2. We **can't store** the value of each state
 - A table housing all state-value pairs would be too huge
- Time for (2)

Too many states!

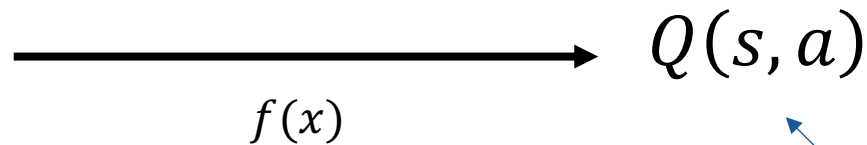
- The problem of many/high-dimensional states is **common in AI problems** — Compare with, e.g., image classification
- Intuitively, the details of each state matter less, the more information there is. There are **similarities**
- As humans, we recognize these similarities, and **disregard** irrelevant information

Function approximation

- Today, we use **machine learning** to learn the value of states
- Instead of storing the Q-value of each node in the tree, we let a function approximate it



x



- It means we can handle larger state spaces!

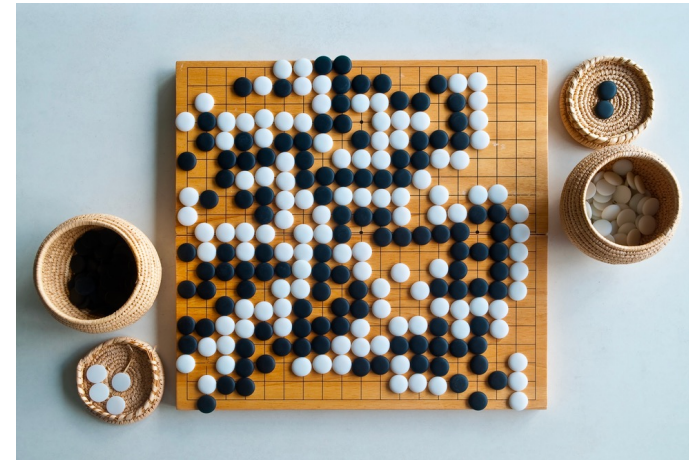
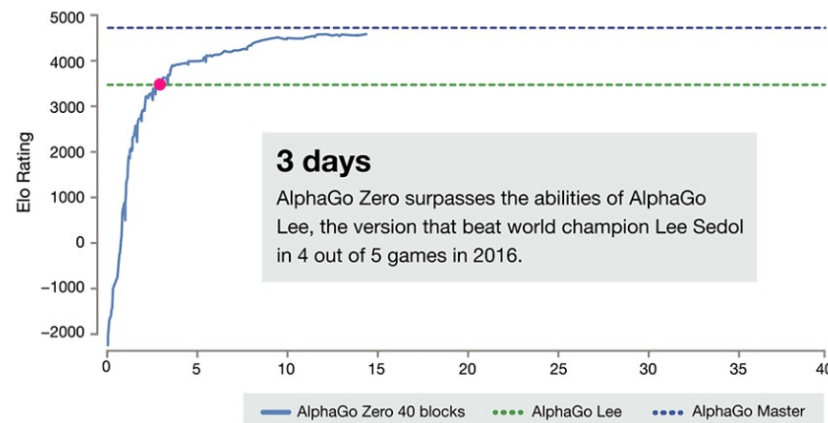
The average reward
after taking action a
in state s

Deep learning

- In modern AI, this function is a **deep neural network**
- During **backpropagation**, we don't just update counters N, Q , but the parameters of neural nets **predicting** Q
- This means we
 - A) Don't have to manually encode the quality of actions
 - B) Don't have to explore every state to predict its value

Self-play

- Crucially, requiring no human-designed value function means agents can learn by **self-play**
- Different versions of the agent play **against each other**



Markov property

- In Chess, Go, Tic-Tac-Toe, the history of a game does not influence transitions—states are **Markovian**

$$p(S_t \mid \underbrace{S_1, \dots, S_{t-1}, A_1, \dots, A_{t-1}}_{\text{All history}}) = p(S_t \mid \underbrace{S_{t-1}, A_{t-1}}_{\text{Most recent}})$$

- If you move a Chess piece from E2 to E4, nothing else happens. The set of valid moves can be read off the board beforehand.

Non-Markov long games

- When Markovianity doesn't hold, **long** games are hard
- We have to **memorize/summarize** history to predict the future
- This is starting to be addressed more by modern systems as well

Take-away

- Use **machine learning** where it helps! Not just “because #ai”
- Machine learning is great at condensing information and extracting relevant patterns
- AlphaGo **combined** traditional MCTS with ML successfully

Next time: **Modern game-playing AI**